

Laboratori de GIA-ABIA

Sessió 1: Cerca no informada

Preparant l'entorn virtual

En aquesta sessió configurarem un directori per fer-lo servir com a projecte pel curs.

Primer de tot, creeu un directori buit. Per utilitzar-ho com a exemple, farem servir el directori ~/abia:

```
1 mkdir ~/abia
```

Després caldrà entrar-hi i crear un entorn virtual. Un entorn virtual és una instal·lació local de Python separada de la del sistema operatiu, i que permet mantenir dependències diferents a la instal·lació del sistema o d'altres entorns virtuals. Això es pot fer amb el mòdul *virtualenv* (recomanable tenir una versió de Python igual o superior a 3.7).

```
1 # Si teniu Ubuntu o Debian:
2 apt-get install python3-virtualenv
3 # Si no teniu Ubuntu o Debian:
4 pip3 install virtualenv
5
6 cd ~/abia
7 python3 -m venv venv/
8
9 # Si teniu Windows:
10 venv\Scripts\activate.bat
11 # Si no teniu Windows:
12 source venv/bin/activate
```

Ara ja podem instal·lar dependències **només** al vostre nou entorn virtual. **Important!** Cada cop que vulgueu treballar amb aquest projecte haureu d'executar sempre l'última comanda (la d'activació).

Un cop activat l'entorn virtual podem instal·lar les dependències per les primeres sessions de laboratori:

```
1 pip3 install --ignore-requires-python aima==2023.2.6 numpy
```

La llibreria *aima* té un nom basat en les inicials del llibre Artificial Intelligence: a Modern Approach de Russell&Norvig i implementa la majoria d'algoritmes introduïts en aquest mateix llibre. Aquesta llibreria té implementacions en molts llenguatges, incloent Python. Nosaltres utilitzarem el modul aima que compila

el codi del repositori <https://github.com/aimacode/aima-python>.

Durant aquest primer bloc de l'assignatura, utilitzarem les classes i funcions incloses al mòdul de cerca d'aima. El codi d'aquest mòdul es pot consultar a <https://github.com/aimacode/aima-python/blob/master/search.py>. Guardeu aquesta URL perquè consultarem aquest codi bastant sovint per entendre com funcionen els algoritmes.

La classe *Problem*

Per resoldre problemes utilitzarem la classe *Problem*, que actua com una classe abstracta que defineix una interfície estàndard per a problemes que es poden resoldre amb algoritmes de cerca. Tot problema que volguem resoldre amb els algoritmes que tractarem hauria de ser formulat com una subclasse de *Problem*, definint els mètodes necessaris.

Podeu trobar aquesta classe a la línia 15 de *search.py* (<https://github.com/aimacode/aima-python/blob/master/search.py>). Per fer cerca no informada, els mètodes que heu de conèixer són:

- **`__init__(initial, goal)`**: Constructora del problema. L'argument *initial* permet especificar l'estat inicial, e.g., en un problema de taulell, aquest podria ser el taulell en la seva configuració inicial. L'argument *goal* és opcional i permet especificar l'estat final del problema, si és només n'hi ha un.
- **`actions(state)`**: Donat un estat, aquest mètode ha de retornar una llista de les accions possibles que es poden realitzar des d'aquest estat. En el context d'un joc de puzzle, podrien ser els moviments legals disponibles.
- **`result(state, action)`**: Agafa com inputs un estat i una acció, i retorna l'estat resultant d'aplicar aquesta acció a l'estat donat. És l'equivalent a moure's a una nova posició en un taulell de joc després d'executar un moviment.
- **`goal_test(state)`**: Donat un estat, aquest mètode determina si aquest estat compleix el criteri per ser considerat un estat objectiu o solució del problema. És recomanable definir els possibles estats finals sobreescrivint aquest mètode a fer-ho amb la constructora, perquè permet especificacions més complexes.

Quan definiu una subclasse de *Problem* per especificar el vostre problema de cerca, s'espera que vosaltres sobrescriviu aquests mètodes per adaptar-los a la naturalesa específica del problema que voleu resoldre.

BFS i DFS a *aima*

Al codi font de *search.py* (<https://github.com/aimacode/aima-python/blob/master/search.py>) podeu trobar implementacions bàsiques de BFS i DFS que només necessiten una instància de la vostra classe *Problem* per executar-se:

- breadth_first_tree_search: BFS sense gestió de repetits
- depth_first_tree_search: DFS sense gestió de repetits
- breadth_first_graph_search: BFS amb gestió de repetits
- depth_first_graph_search: DFS amb gestió de repetits

Llegiu amb atenció el codi d'aquestes quatre funcions i observeu les diferències. S'assemblen als algorismes que heu vist a teoria?

Com es fa la gestió de repetits? Quan dirieu que és millor utilitzar una versió amb i quan una versió sense gestió de repetits?

Exemple de problema: *Problema 5*

Ara veurem un exemple de Problem, que resol el Problema 5 de cerca heurística de la col·lecció de problemes:

```

1 from typing import List
2 from aima.search import Problem, breadth_first_tree_search
3
4 class Problema5(Problem):
5     def __init__(self, initial_state=None, goal_state=None):
6         if initial_state is None:
7             initial_state = ['A', 'R', 'A', 'R', 'A']
8         if goal_state is None:
9             goal_state = ['R', 'R', 'R', 'A', 'R']
10
11         if len(initial_state) != len(goal_state):
12             raise Exception("Length of arguments should be equal")
13
14         super().__init__(initial_state)
15         self.goal_state = goal_state
16
17     @staticmethod
18     def flip_coin(coin: str):
19         if coin == 'A':
20             return 'R'
21         elif coin == 'R':
22             return 'A'
23         else:
24             raise Exception("Coin is not valid, should be A or R")
25
26     def actions(self, current_state: List[str]) -> range:
27         return range(len(current_state))
28
29     def result(self, current_state: List[str], next_action: int)
30         -> List[str]:
31         new_state = current_state.copy()
```

```

31     pos_1 = next_action
32     pos_2 = (next_action + 1) % len(current_state)
33
34     new_state[pos_1] = self.flip_coin(current_state[pos_1])
35     new_state[pos_2] = self.flip_coin(current_state[pos_2])
36
37     return new_state
38
39 def goal_test(self, current_state):
40     return current_state == self.goal_state
41
42
43 if __name__ == '__main__':
44     example_initial_state = ['A', 'R', 'A', 'R', 'A']
45     example_goal_state = ['R', 'R', 'R', 'A', 'R']
46
47     problem = Problema5(example_initial_state, example_goal_state
48                          )
49     node = breadth_first_tree_search(problem)
50     print(node.solution())

```

Analitzeu aquest codi, mirant amb atenció l'enunciat 15, i intenteu entendre com aquesta implementació resol el problema.

Fixeu-vos sobretot en com aquest codi genera les accions possibles. Com que el problema és senzill, n'hi ha prou amb identificar posicions concretes del tauler i suposar que permutem la fitxa d'aquesta posició amb la fitxa de la posició contigua, sempre a la dreta.

Amb problemes més complicats, pot ser que la representació de les accions possibles sigui tantmateix més complicada també. Això ho veureu en altres problemes i especialment en la pràctica.

El mateix succeeix amb l'estat del problema. Com en aquest cas el tauler és senzill, n'hi ha prou amb tenir una llista de N posicions. Què passaria si tinguéssim una combinació d'elements, com per exemple si apart de la llista haguéssim de guardar puntuacions en una altra estructura com una matriu?

Una possible manera de solucionar aquests problemes és crear classes pròpies específiques del problema per a representar estats i accions. Aquesta setmana veurem classes per representar accions i la setmana vinent veurem com afegir classes per representar estats.

Següents passos

1. Modifiqueu la implementació del problema 5 per a representar les accions amb una classe que anomenarem Flip:

```

1 class Flip(object):

```

```
2     def __init__(self, pos_1: int, pos_2: int):
3         self.pos_1 = pos_1
4         self.pos_2 = pos_2
5
6     def __repr__(self):
7         return f"Flip coins: {self.pos_1} and {self.pos_2}"
```

2. El codi que heu provat utilitza BFS sense gestió de repetits. Proveu ara amb DFS sense gestió de repetits (`depth_first_tree_search`). Què succeeix? Podeu interpretar què està passant?
3. Proveu qualsevol dels dos algoritmes amb gestió de repetits:
 - `breadth_first_graph_search`
 - `depth_first_graph_search`

Què succeeix? (**Nota:** en la següent sessió plantejarem una sol·lució.)

4. Resoleu el problema 15 de la col·lecció de problemes (cerca heurística), amb classes per representar les accions possibles i proveu tots els algoritmes.